Mubit Engineering Control

Internal implementation plan version 3 with designed agents and continuous evaluation

Prepared for Shankha Dutta and the Mubit engineering team | 2 November 2026

The product we should build

Build one private web application that turns an implementation plan into coordinated work across the team's Claude Code, Codex, and OpenCode installations. Employees choose which of their accounts and machines can run work. The system proposes a task breakdown, schedules independent tasks, presents questions and approvals, and produces a tested, reviewable result against the company repository.

Use existing coding agents as execution engines. A task architect designs specialised subagents from the work, a policy compiler validates their definitions, and a broker runs them on authorised employee profiles. Capture their execution, evaluate the result and coordination, and let an improvement agent propose better versioned designs through controlled experiments.

What the team gets

- One project workbench for plans, designed agents, dependency graphs, sessions, diffs, decisions and evaluations.
- Task-specific subagents with explicit scope, context, tools, output contracts and evaluation criteria.
- Employee-owned Claude Code, Codex and OpenCode execution profiles on Linux VMs and optional Mac workers.
- Traceable workflows and a measured improvement loop for agent instructions, context and coordination.
- An integration branch and PR whose combined changes passed the required checks.

Initial operating assumptions

Size the first release for a planning scenario of 4 to 10 engineers, 3 enrolled employee runners, 6 active task workers, and one GitHub organization. These are adjustable design assumptions, not measured team facts. Begin with private repositories, Linux execution, and one repository per delivery; add macOS and coordinated multi-repository work after the first pilot.

Estimate 100 to 140 engineer-days for the expanded internal release, including the original platform, dynamic agent design, observability, evaluation harnesses and controlled improvement. Plan roughly 9 to 13 calendar weeks with three experienced engineers, or 13 to 19 with two. A small delivery pilot arrives earlier; meaningful improvement claims require additional representative task evidence. Optional GitHub source-driven intake adds 12 to 20 engineer-days; begin the first pilot with manual plans.

1 Product boundaries and success criteria

The primary user is an engineer coordinating an implementation that spans several independently reviewable pieces. The CTO or delivery owner approves the overall plan, while task owners and designated reviewers make the decisions within their scope.

Required behaviour

- Accept a repository, pinned base, implementation plan, constraints and acceptance checks as a delivery.
- Propose tasks and design or reuse the subagents each task needs, with explicit output and evaluation contracts.
- Run independent work on authorised employee profiles and isolated task checkouts.
- Expose the workflow, subagents, messages, decisions, checks and observability coverage to the team.
- Capture immutable agent and workflow versions, parentage, account ownership and execution evidence.
- Evaluate agent output, coordination and the combined delivery; preserve work through failures.
- Use a separate improvement agent to propose changes, run experiments and promote only eligible versions.

Scope decisions for the first release

Implement one internal organization, existing company identity, GitHub integration, and a responsive browser UI. Start with polling plus durable events rather than a separate distributed workflow product. Expose a small workflow configuration format with fixed step types; defer a general visual workflow builder until recurring usage justifies it.

Treat hosted multi-tenancy, customer billing, an agent marketplace, native phone apps, model training, and public hosting as later independent projects. Employees can keep using the vendor terminal interface through the web workbench when a structured adapter cannot yet represent a particular interaction.

Release acceptance

Three employees can run a plan through dynamically designed subagents using all three CLI types, answer a shared decision, recover an interrupted run and review a tested integration PR. Every managed child is registered and visible. An evaluation experiment must compare a candidate workflow with the incumbent, record its evidence and demonstrate a controlled promotion or rejection.

Measure elapsed delivery time, human review time, integration repair time, and escaped defects against the team's current workflow. More concurrent terminals is not itself a success measure.

2 Features to adopt and what to build ourselves

Source	Patterns to adopt	Our implementation
Cezar [1, 2]	Parallel task worktrees, workflow stages, child dispatch, compare attempts.	Fixed workflow steps backed by a task graph, explicit gates, and durable attempt records.
Kandev [3, 4]	Project boards, workbench, mixed agents, executor profiles, team access.	One project-first UI with native account ownership and shared task permissions.
Happier [5, 6]	Remote session access, view and control grants, delegated approvals.	Private session sharing, one active terminal controller, and scoped decision grants.
Orchestrator.inc [7, 8]	Planner and workers, PR visibility, CI and review feedback routed to workers.	A coordinator proposes work; GitHub events route to the attempt that owns each branch.
DeepSeek prototype [35]	Event intake, scoped work, follow-ups and prompt upgrades.	Optional admission layer; existing tasks, employee grants and pinned agents.

These are interaction and architecture references. Our custom design adds a versioned agent factory, independent evaluation and an evidence-driven improvement loop to the referenced interaction patterns. Experimental features in those projects are useful evidence of direction, not a production dependency.

The custom core

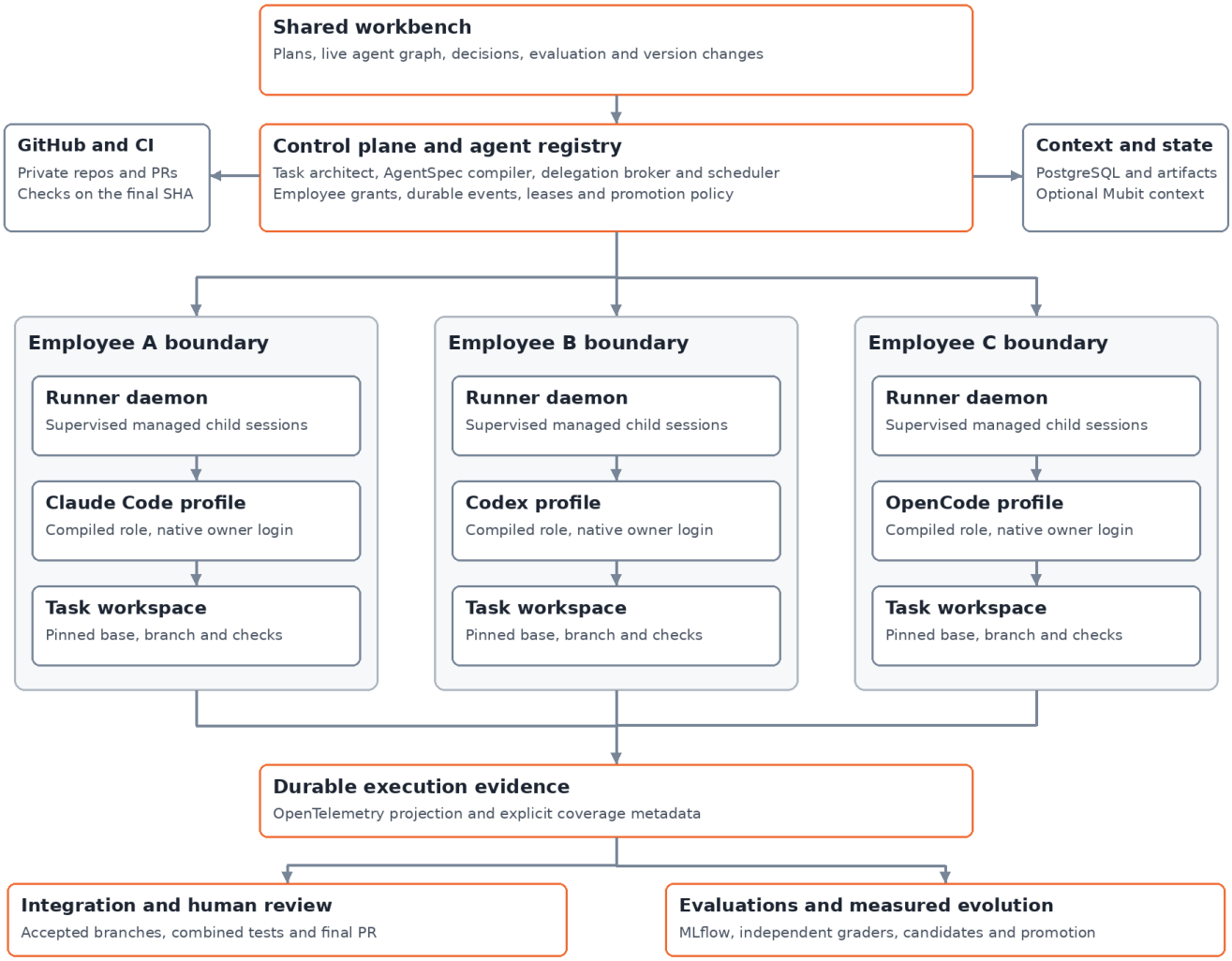
- Employee-owned execution profiles with separate grants for viewing, prompting, account use, tool approval, and merging.
- Validated task dependencies, pinned input commits, explicit child budgets, and bounded delegation.
- Durable leases and recovery that prevent duplicate authoritative attempts.
- An integration queue that validates the actual combination of accepted changes.
- A decision ledger that binds approvals to the exact plan or operation reviewed.
- Capability-aware adapters that state which actions each installed agent version can support.

Build strategy

Create one TypeScript monorepo and implement the core independently. Use the reference products to test flows and compare design decisions. At the end of the initial spike, reconsider using a single upstream foundation only if it passes the account, recovery, approval, and integration acceptance tests with less total maintenance.

Avoid combining several application databases or session models. For any actual source reuse, record the repository, file, commit, and applicable license in a provenance manifest before import. The referenced main repositories currently list MIT for Cezar and Happier, AGPL-3.0 for Kandev, and Apache-2.0 for Orchestrator.inc [1, 3, 5, 7].

3 End to end architecture



Proposed complete architecture. Agent definitions are compiled before execution. Native credentials stay with their owners; traces and evaluation records stay on company infrastructure.

The control plane owns plans, agent definitions, grants, task leases and promotion policy. A generated agent definition selects permitted capabilities; the broker binds it to an existing employee execution profile and runs a managed child session.

Execution evidence flows through the durable event stream into OpenTelemetry and self-hosted MLflow. Independent evaluators score outputs and coordination. A separate improvement agent proposes candidate definitions and workflow changes; an experiment and promotion controller decides which version becomes eligible for future work.

GitHub and CI verify the combined commit. Mubit optionally supplies dated repository context and approved lessons. The telemetry backend is replaceable and is not the authority for permissions, leases or acceptance.

4 Software architecture and repository layout

Use a modular control plane, separate runner daemons, and an isolated evaluation worker service. Keep domain state transitions in ordinary tested functions, with storage and provider adapters behind narrow interfaces.

Area	Initial choice	Reason
Web	React, Vite, TypeScript	Fast workbench UI; no server rendering requirement.
API	Node active LTS, Fastify, Zod	Shared contracts; explicit request validation.
Storage	PostgreSQL and SQL migrations	Transactions, leases, constraints, and audit state.
Queue	PostgreSQL job tables and outbox	One durable source; no initial Redis dependency.
Runner	TypeScript, child processes, PTY support	Shared schemas and agent SDK ecosystem.
Live UI	SSE; WebSocket for terminal and runner	Replayable events; bidirectional control where needed.
Artifacts	Private object store; runner disk for workspaces	Large logs and screenshots stay out of database rows.
Identity	Company OIDC and server sessions	Reuse employee identity and offboarding.

Module boundaries

```
apps/web                project workbench and decision inbox
apps/control            API, scheduler, outbox, reconciliation
apps/runner             process supervisor and local journal
packages/contracts      schemas and protocol versions
packages/domain         task, plan, approval and lease rules
packages/agents         Claude, Codex and OpenCode adapters
packages/git            workspaces and integration operations
packages/policy         grants and executable policy checks
packages/testkit        fake runners and recorded protocol fixtures
```

Add an agent registry and compiler, telemetry projection, dataset service, experiment coordinator and promotion controller. The evaluation worker uses a small pinned Python environment for MLflow and optional GEPA integration. Keep orchestration database access in the control service. A runner receives scoped run instructions, never unrestricted database credentials. Bootstrap with a containerised web/API service, PostgreSQL, and a runner installed as a system service. Pin dependency and agent versions after the compatibility spike; verify upgrades against fixtures before promotion.

Use row locks and unique constraints for task claims. PostgreSQL documents the locking primitives; our lease and reconciliation protocol supplies application-level ownership semantics [15]. Add a dedicated workflow engine only if long-running cross-service transactions later exceed this model.

5 Employee accounts and permissions

Represent an employee, a coding agent, an agent account, and a runner as different entities. An execution profile binds them together for a permitted use. A generated AgentSpec is separate: it defines the job, never creates a vendor account or grants access. The employee's dashboard identity is never treated as proof that a matching vendor account is authenticated.

```
ExecutionProfile
  id, owner_user_id, agent_kind, agent_version
  account_ref, runner_id, runtime_kind, capability_set
  allowed_project_ids, permitted_operator_ids
  max_active_attempts, auth_state, availability
```

Credential handling

Have the owner complete the vendor's native sign-in flow inside their persistent runner boundary. The control plane stores an opaque account reference, a display label, supported identity metadata, and authentication health. It does not collect employee passwords or export subscription tokens into a central account pool. API-backed service profiles are a separate explicit configuration.

Begin with one persistent Linux VM per employee, hosting their authorised agent installations. A VM can run several task worktrees, subject to per-profile limits. A central host administrator remains trusted. For stronger task isolation, introduce per-task containers or VMs without giving agents access to the host container socket or other employees' credentials.

Permission	Default holder	Scope
Observe a session	Project members	Transcript, logs, diff; sensitive fields redacted.
Prompt or steer	Task owner and collaborators	This task and attempt.
Use an account profile	Owner and named delegates	Approved projects and bounded runs.
Approve tool execution	Owner or explicit delegate	Exact pending operation.
Approve a plan	Delivery owner	Exact plan revision.
Merge a PR	Authorised GitHub reviewers	Repository branch rules still apply.

Viewing a session does not grant shell input, account use, or permission approval. Global app administrators may manage the installation but do not automatically inherit approval authority for a person's account. Host-level administrators remain able to access host resources; show that operational trust boundary accurately.

Keep native Claude subscription use inside the unmodified owner CLI. Use supported API credentials for SDK-backed service profiles. Anthropic's current documentation distinguishes these authentication paths; do not infer that an internal dashboard authorises credential pooling [11].

6 Runner service and execution environments

A runner is a persistent service responsible for processes, workspaces, local credentials, event journaling, and reconnection. It is not an LLM. A coordinator cannot create a new runner, change its permissions, or install new hooks through an ordinary task prompt.

1. An administrator creates a short-lived, single-use enrolment invitation for a specific employee and environment.
2. The runner exchanges the invitation for a device identity and receives project-scoped authorisation. It reports OS, architecture, agent versions, disk and process capacity, and supported adapter actions.
3. The employee signs in to their native agents. A compatibility probe verifies each selected profile without printing credentials.
4. The runner maintains an outbound TLS connection to the control plane. Devices can live behind a VPN or NAT; no vendor agent server needs to be publicly reachable.
5. For a leased attempt, it prepares the pinned repository, starts the selected agent, and journals output before forwarding events.
6. On completion it preserves the branch, captures checks and artifacts, stops owned background processes, and releases capacity.

Filesystem and process rules

Use one repository clone per employee boundary and one worktree per task attempt. Git worktrees on different hosts use independent clones; never assume a local worktree path exists on another runner. Cache dependencies within the same employee boundary, key build caches by toolchain and lockfile, and allocate unique service ports per attempt.

Start processes with argument arrays and validated paths. Treat prompts, branch names, and repository URLs as data rather than shell fragments. Record supervisor ownership using an attempt ID and generation, not only a PID. Stop uses the matching process group or job container, then confirms termination before making capacity available.

macOS support

Use a Mac mini runner when the repository requires Xcode, signing, or another macOS dependency. Run the service in the intended OS account and verify keychain access after restart. Do not assume a login on the operator's laptop authenticates a headless macOS service. Linux remains the initial release target; validate macOS stop, resume, keychain, and filesystem behaviour in its own milestone.

Control plane outages

A disconnected runner stops launching new work and buffers events locally. It blocks new operations that require a human decision. After its lease grace period it parks the session when the adapter supports it, otherwise interrupts it and preserves the workspace. Existing external side effects may finish; reconciliation must inspect them before any replacement attempt starts.

7 Agent adapters and version compatibility

Every adapter implements one contract and reports its capabilities, including native-child discovery, lifecycle events and child-level usage. A uniform UI must not imply that all agents can pause, resume, fork, approve, or change model in the same way. Native terminal access remains the fallback for unsupported interactions.

Agent	Preferred integration	Important boundary
Claude Code	Native CLI streaming plus documented hooks; PTY fallback.	Owner signs in natively. Prove structured questions and approvals in the pinned version [9, 10, 11].
Codex	Local app-server subprocess over stdio; PTY fallback.	Current docs call app-server experimental. Pin the binary and generated types; keep a native fallback [12].
OpenCode	Local server API and event stream for the installed major version.	V1 and V2 differ. V2 permission routes are experimental; test the chosen version [13, 14].

Common interface

```
probe(profile) -> capabilities and auth health
start(attempt, inputs) -> native_session_ref
send(native_session_ref, message, command_id)
interrupt(native_session_ref, command_id)
resume(native_session_ref, checkpoint, command_id)
resolveDecision(native_request_ref, decision)
attachTerminal(native_session_ref, controller_lease)
collectResult(attempt) -> branch, checks, artifacts
```

Advertise structured_stream, native_terminal, questions, tool_approval, interrupt, native_resume, usage, and session_fork separately. If a capability is absent, show a clear unsupported state and offer the compatible path. A native session ID is scoped to its runner, agent version, and authentication profile; it is not portable by default.

Approval bridge

Map native request IDs to a single durable platform approval. For Claude Code, test documented permission and tool hooks against the installed version, including question answers and plan exits. Do not infer approval by parsing English terminal output. For Codex, handle server-initiated protocol requests. For OpenCode, use that major version's request/reply schema.

When a native process or hook times out, expire its platform request and verify whether the tool ran. A later browser click cannot revive a stale request. SDK-based Claude service operation uses an explicitly supported credential profile rather than silently reusing a subscription session.

Keep replayable redacted protocol fixtures, a tiny real repository test, and one canary runner per adapter version. Upgrade individual profiles after compatibility tests; do not force a fleet-wide agent update during active deliveries.

8 Planning and automatic task decomposition

The coordinator is a normal selected coding agent with read access and scoped task-management tools. It proposes work; deterministic code validates the result and the delivery owner approves the revision. The initial plan can be supplied in full, so decomposition must preserve its requirements rather than rewrite its intent.

Planning procedure

1. Pin the repository commit and read its instructions, package boundaries, test commands, and existing interfaces.
2. Extract requirements and acceptance criteria into a traceable list. Mark unresolved decisions and any missing evidence.
3. Identify prerequisites such as schema, API contracts, shared types, and migration ordering.
4. Split the remaining work into reviewable tasks with bounded file ownership, dependencies, expected output, validation commands, and a suggested profile.
5. Estimate concurrency from dependency independence and available resources. Flag tasks likely to edit the same files.
6. Validate the graph, then present the tasks, dependencies, assignments, and unresolved choices for human approval.

Task specification

```
task_id, requirement_ids, plan_revision
objective, deliverables, acceptance_checks
depends_on[], base_policy, pinned_base_sha
owned_paths[], shared_contract_refs[]
execution_profile_id, reviewer_profile_id
max_attempts, time_budget, concurrency_group
completion_artifact, integration_order
```

Graph validation rejects cycles, missing dependencies, references outside the delivery, missing acceptance criteria, and unavailable profiles. Path overlap is a conflict signal, not a complete semantic dependency detector. Lockfiles, migrations, generated types, route registries, and shared configuration need explicit coordination even when the planner labels the work independent.

Dependent branches

A task that depends on code waits for its prerequisite to be accepted into the delivery integration branch. It starts from the resulting pinned integration commit. A task that needs only an approved interface can start against a versioned contract and later validate against implementation. Record which policy was chosen; do not mark a code dependency satisfied merely because its agent has stopped.

The architect submits child tasks and AgentSpec definitions together. The compiler validates both before broker dispatch. Start the pilot at one child level; permit a depth of two only after the nested ownership and evaluation tests pass. Automatic splits stay inside the approved scope, grant, fan-out and remaining budget.

9 Task specific agent design

A task architect must be able to invent a useful specialist for the current work, rather than choose only from a fixed list of job titles. The output is a typed AgentSpec containing instructions and constraints. It reuses an existing model and runtime; creating a subagent does not train a new model or create a new employee account.

Design procedure

1. Inspect the task, repository boundaries, approved contracts, dependencies and known failure patterns.
2. Choose whether a specialist adds value. Keep the work in one agent when shared context or integration overhead dominates.
3. Reuse a suitable proven definition or draft a task-specific role with explicit inputs, output schema and checks.
4. Select required tools from the registered catalog and propose a compatible authorised execution profile.
5. Compile the definition, validate the handoff and reserve its budget through the scheduler.
6. Register the instance and its parent before the first tool or model call can run.

Definition contract

```
AgentSpec
  id, revision, content_hash, creator, purpose
  role_instructions, task_input_schema, output_schema
  tool_capability_refs, context_policy, skill_refs
  allowed_runtime_kinds, model_constraints
  workspace_scope, runtime_policy_ref, checks
  eval_contract_ref, spawn_policy, budget
  source_plan_revision, reusable_status
```

For a schema change, the architect may define a migration author, a compatibility reviewer and a rollback checker. For a contained function fix, one implementer and the ordinary validation path may suffice. Generate only the team that the task warrants.

A compiler checks schema validity, tool registration, supported adapter actions, output contracts, profile eligibility and the approved policy ceiling. Invalid proposals return actionable errors. Bounded repair can revise the proposal; repeated failure becomes a team decision.

Anthropic reports that tightly coupled coding tasks can offer less useful parallelism than research [22]. Its later coordination work also motivates explicit contracts and measuring group behaviour [24]. Keep a strong unsplit baseline and allow the designer to remove unnecessary roles.

10 Delegation execution and native subagents

Make platform-managed children the canonical path. Each child is a real registered session, bound to an employee profile and a task workspace. That works across different CLIs and does not depend on a vendor supporting cross-vendor subagents.

Mode	How it runs	Requirement
Managed child	Broker launches a separate native CLI session.	Own instance ID, lease, trace, budget and result contract.
Native child	The selected CLI launches its own internal specialist.	Register native IDs and capture supported child events; verify limits.
Opaque native activity	Native work is visible only through terminal output or a parent summary.	Mark incomplete coverage; exclude from evidence requiring child-level measurement.

Claude Code, Codex and OpenCode expose custom agent concepts, but their configuration and permission inheritance differ [32, 33, 34]. Our compiler emits a pinned, session-scoped configuration only when the adapter proves it can preserve platform policy. A native role file is not automatically an access-control boundary.

Broker guarantees

- Validate child capabilities against the parent authorisation envelope, project policy, owner grant and runner boundary; reject privilege expansion.
- Reserve child runtime and usage from the parent budget. Enforce depth, fan-out, total descendants and the existing global worker limit.
- Require a parent and task ID, creation reason, expected output, deadline and cancellation behaviour.
- Use the existing lease generation, idempotency and recovery protocol for every managed child.

Start with maximum fan-out 3, pilot depth 1 and at most 12 child instances per delivery as proposed tunable defaults. Native descendants must count toward the same limits. Where the installed CLI cannot expose or constrain native spawning reliably, disable that mode for controlled workflows and use managed children.

Avoid queue deadlock: a parent waiting on children releases its execution slot after a durable checkpoint, or the scheduler reserves child capacity before launching a blocking parent. Cancellation propagates and waits for acknowledgement; intentional orphan work needs a new recorded owner.

11 Scheduling ownership and state transitions

The scheduler is deterministic. It selects tasks whose prerequisites and permissions are satisfied, then atomically reserves the task attempt and its profile capacity. An LLM may recommend priorities; it does not grant leases or bypass limits.

Object	Normal progression	Other states
Task	Proposed, approved, ready, active, awaiting review, accepted, integrated.	Blocker reason records dependency, capacity or decisions.
Attempt	Queued, leased, preparing, running, work complete.	Awaiting input, interrupted, quota wait, failed or cancelled.
Delivery	Planning, approved, executing, integrating, awaiting merge, merged.	Cancelled; retain accepted work and evidence.

Use separate reason fields for blockers such as dependency, capacity, authentication, quota, policy, and reviewer availability. A finished attempt is work_complete; only independent checks and review can make the task accepted. Merged is confirmed by the repository host.

Selection and fairness

Apply delivery priority first, then fair rotation across employees and projects. Enforce global, runner, employee, profile, and delivery limits. Start with 6 active attempts globally, 2 per enrolled employee, and 1 per agent account profile. Include designers, coordinators, reviewers, native descendants and improvement jobs in these limits. Reserve execution for required integration and evaluation work; run speculative improvement at lower priority. Increase profile concurrency only after native login refresh and quota behaviour are tested.

Durable leases

Claim work in a database transaction with an attempt ID, generation, expiration, and a unique active slot for the task. Use a 10-second heartbeat and an initial 60-second lease as tunable defaults. A claim and its dispatch outbox record commit together. Every runner command and event carries the attempt generation and an idempotency key.

Lease expiry alone does not prove the old process is dead. Reconcile runner ownership first. Block or quarantine replacement writers until the previous attempt is stopped or isolated. Stale generations cannot publish accepted results, update task state, push an authoritative branch through the managed path, or request an integration operation.

Quota and retries

Known exhaustion moves the attempt to quota_wait and preserves its workspace. An observed reset time can make it eligible later; unknown quota stays unknown. Do not rotate to another employee's account automatically. A deliberate reassignment creates a handoff and a fresh attempt. Bound transient retries and repair loops; approval rejection and authentication failure require a different resolution from a network timeout.

12 Shared workbench and team decisions

Use a project-first layout with compact navigation, terminal-friendly typography, uppercase status labels, and a restrained orange accent. Keep the primary screen focused on the current delivery and items that need a human response.

Surface	What it shows	Primary actions
Project overview	Deliveries, active workers, blockers, profile availability.	Create delivery, inspect blocked work.
Plan and graph	Requirements, tasks, dependencies, plan changes.	Edit, approve revision, change assignments.
Task workbench	Conversation, terminal, diff, files, checks, preview.	Message, interrupt, retry, review result.
Decision inbox	Question, evidence, affected task, authorised deciders.	Answer, approve, reject, request revision.
Integration review	Accepted branches, combined diff, CI evidence.	Send back, include task, create or review PR.
Fleet and accounts	Owner, agent, runner, authentication, capacity.	Enrol, test, grant, drain, revoke.
Agent graph	Parentage, role definitions, messages, context and budgets.	Inspect child, steer, cancel, compare versions.
Evals and changes	Scorecards, experiment cases, candidates and rollout state.	Inspect evidence, approve, reject, revert defaults.

Collaboration rules

Several people can read a session simultaneously. Messages enter one ordered queue and record their authors. Show queued versus delivered messages, and do not inject a steering instruction twice after a reconnect. Give native terminal input to one controller at a time using a short renewable lease; observers can request or be granted takeover.

Plan edits use optimistic version checks. If two people edit the same revision, return a conflict with the newer text. An approved plan is immutable; edits produce a new revision with an impact summary. Independent tasks can continue when their inputs are unaffected. Pause and invalidate approvals only for the affected tasks.

Decision card contents

Show the exact question or requested operation, relevant diff or command, requesting agent, account owner, task, expected consequence, and supporting evidence. Present choices in plain language. Keep the technical detail available beside the decision rather than forcing the reviewer to search the transcript.

Keep plan approval, tool approval, result acceptance, agent-definition reuse and workflow promotion separate. A click in one category grants no authority in another. Notifications link to the underlying decision, exact version and evidence. Start with the web inbox and optional browser notifications.

13 Approval integrity and executable policy

Keep approval policy in deterministic code. The planner may explain a proposed action, but cannot approve itself, broaden its account grant, or redefine a protected workflow. A reviewer agent can recommend acceptance; a human decision remains attributed to the actual person.

Approval record

```
Approval
  id, kind, project_id, task_id, attempt_id, generation
  native_request_ref, plan_revision, policy_revision
  operation_hash, input_commit, allowed_decider_ids
  status, expires_at, resolved_by, resolved_at, answer
```

Resolve a pending approval with an atomic compare-and-set. The first authorised answer wins; a concurrent answer sees the recorded result. Bind tool decisions to the native request and complete tool input. Bind plan and merge decisions to the reviewed revision or commit. Changes to that input make the earlier decision stale.

Execution modes

Mode	Behaviour	Intended use
Observe	Read and analyse; no implementation changes.	Planning and review.
Implement	Allow scoped edits and approved test commands.	Normal task workers.
Assisted	Surface additional supported tool decisions.	Sensitive or unfamiliar work.
Unattended bounded	Preapproved task scope, runtime and tool rules.	Routine work after a successful pilot.

Modes are capability-dependent. Report what the adapter and runtime actually enforce. Owned paths in a task brief are coordination instructions unless backed by filesystem restrictions; never label them a sandbox. Isolation must come from the VM, container, mounted paths, network rules, or native agent controls.

Timeout and revocation

Never convert an unanswered question or an expired approval into consent. If the native request can remain pending, preserve it; otherwise interrupt or reject and reopen work through a supported path. Revoke account-use grants immediately for new work, stop or drain affected active work according to the owner's choice, and expire pending approvals whose authority has changed.

A project pause prevents new dispatch. A fleet stop sends interruption requests to every current attempt, tracks acknowledgements, and identifies unreachable runners. It cannot truthfully claim to have stopped a disconnected machine until the runner confirms or host administration verifies it.

14 Repository workspaces and integration

The source repository is canonical. Every attempt records its base SHA, branch, produced commits, and evidence. Git worktrees provide separate working directories sharing repository data within a clone [16]. They do not eliminate merge conflicts, isolate processes, or coordinate semantic changes.

Workspace preparation

1. Verify the repository URL and permitted project; fetch the approved remote and pin the selected base commit.
2. Create an attempt branch and worktree. Use stable identifiers in paths and branch names.
3. Resolve required private submodules recursively using repository-scoped credentials. Record submodule SHAs; fail preparation when required content is unavailable.
4. Run a reviewed repository bootstrap profile. Record toolchain versions and dependency installation results.
5. Provide the task brief, approved contracts, relevant decisions, and validation commands. Allocate unique ports and test database namespaces.

Integration queue

Create a delivery integration branch from the pinned repository base. Before accepting a task, verify its commits and checks. Integrate accepted branches in dependency order, then run the relevant checks on the exact resulting integration SHA. Use a single integration lease per repository delivery.

When a merge conflicts, create a bounded repair task against the integration workspace with both changes and their requirements. Do not silently discard one side or restart the entire delivery. Re-run affected checks and invalidate result approvals tied to the old SHA. A later task can use the newly accepted integration commit as its base.

The final PR targets the repository's normal protected branch. Track source branch updates and target branch movement. If main advances before merge, update the integration branch and run the required checks again. Use the repository's merge queue where configured; confirm merge via GitHub rather than assuming an API request completed.

Multiple repositories

Add one task graph spanning repositories after the single-repository pilot. Pin every repository and shared interface version separately. Produce one PR per repository with a delivery manifest describing compatibility and merge order. Cross-repository merges are not atomic. Use backward-compatible API and schema changes and explicit rollout ordering where that matters.

Use a GitHub App for narrowly scoped platform operations and CI observation. Keep employee Git identity distinct from the bot identity. Platform task approval does not manufacture another GitHub human reviewer or satisfy branch protection that requires one.

15 Validation review and completion evidence

An agent ending its turn is not a successful implementation. The platform records separate states for implementation complete, checks passed, review accepted, integrated, and merged. Each state requires its own evidence.

Repository validation profiles

Store reviewed command profiles for bootstrap, lint, type checks, unit tests, integration tests, build, and browser checks as appropriate to each repository. A task chooses relevant checks; the final integration profile covers the combined change. Preserve structured results, exit status, logs, duration, environment identity, and tested commit.

Capture the test runner's output directly. Distinguish a failing test from a test system that could not start. Give an implementation repair attempt the exact failures and remaining budget. Stop after the configured number of repairs and ask for a human decision with the evidence attached.

Independent review

Choose a reviewer profile independently from the implementing profile when possible. Give it read access to the relevant commits, requirements, and test results. Keep review prompts focused on correctness, missing behaviour, interfaces, data migrations, and regressions. Store findings against the reviewed SHA; route actionable feedback to the task owner.

Reviewer agreement is a recommendation. Human acceptance records the reviewer and exact result. When new commits are added, retain the history but require review of the changed evidence. Avoid using two model votes as a substitute for deterministic checks or repository-required human approval.

Browser previews and competing attempts

Provide a private preview for web tasks with a unique port, authenticated proxy, and an explicit lifecycle. Browser tests can attach screenshots and traces. Treat rendered pages and repository text as untrusted inputs to the agent; they cannot change platform authorisation.

Later, allow two or three attempts for the same task when the delivery owner requests comparison. Keep their branches separate, apply the same acceptance checks, and let the reviewer select one. Archive the alternatives and their evidence. This feature consumes ordinary profile and budget capacity; it must not multiply workers outside the scheduler.

Definition of done

- Every requested requirement has an implemented change or an explicit accepted exclusion.
- Task and integration checks are attached to the relevant SHAs.
- Review findings have a disposition and required people have approved.
- The final PR includes a clear change description, validation results, migration notes where relevant, and links to the delivery decisions.

16 Observability across the full workflow

Observe both what the agents did and how the orchestration behaved. The durable application event log remains authoritative. An asynchronous projection exports OpenTelemetry spans to self-hosted MLflow; the product UI joins traces back to tasks, decisions and exact artifacts.

Trace structure

Create bounded traces for planning, each execution attempt, validation, integration, evaluation and improvement experiments. Correlate them with delivery_id, task_id, agent_instance_id, parent_instance_id and workflow_revision. Use span links for dependencies, joins and asynchronous handoffs, since a delivery graph is not a single parent-child tree [25, 26].

Operation	Capture
Design and compile	Source task, AgentSpec revision, validation failures and selected capabilities.
Agent invocation	Profile owner, harness version, model, context manifest and session reference.
Delegation and messages	Sender, recipient, reason, dependency, delivery status and structured artifact refs.
Tools and checks	Visible inputs and outputs, execution status, time and tested commit.
Waiting and integration	Queue, quota, approval and dependency waits; merged inputs and resulting SHA.
Evals and changes	Dataset, scorer, candidate and baseline versions; score evidence and promotion decision.

Coverage and reliability

Record instrumentation coverage explicitly: lifecycle, visible messages, tools, native children, model calls and usage. CLI adapters expose different subsets. Do not infer private reasoning, inaccessible model requests or precise cost from a terminal transcript. Missing data is unknown, not zero.

Normalise native events once, retain their source IDs and journal them before export. Deduplicate reconnects, preserve causality and flag gaps. Pin the evolving GenAI semantic convention version and map stable mubit.* fields independently. Prefer spans whose meaning matches the standard; local definition compilation can use a custom span rather than pretending to be a remote create-agent call.

Retain all critical lifecycle, decision and evaluation records at pilot scale. Apply redaction before export. Capture large content through access-controlled artifact references. Telemetry export failure must not lose the orchestration journal or block unrelated work; sustained coverage loss blocks promotions that require that evidence.

17 Evaluation at agent and workflow level

An evaluation contract states what counts as success before the agent runs. Score execution evidence and repository state, not a worker's own confidence. Use deterministic checks where possible, calibrated model judges for qualitative criteria, and human review for ambiguous or consequential results [23].

Level	Main question	Evidence
Agent output	Did this specialist fulfil its exact contract?	Schema validity, relevant tests, correct patch or evidence-backed review.
Handoff	Was the next agent given usable, consistent inputs?	Contract references, context sufficiency, missing assumptions and message acknowledgements.
Coordination	Was the work split and ordered effectively?	Duplicate edits, loops, wait time, missing dependencies and integration repair.
Delivery	Did the combined change solve the requested problem?	Independent acceptance tests on the integrated SHA and human disposition.
Operations	Was execution dependable and proportionate?	Recovery, resource use, account health, elapsed time and intervention time.
Improvement	Did the candidate outperform a fair incumbent?	Paired experiments, regression checks, uncertainty and prospective outcomes.

Decision rules

Treat correctness and required authorisation as constraints. Compare verified completion, regressions, reviewer effort and integration repair before optimising latency or usage. Preserve the score vector and evidence; a blended average must not hide a failure on a critical requirement.

Pin grader code, rubric, judge model, input artifacts and evaluation-contract revision. Model judges emit structured findings with evidence references and an uncertain or abstain outcome. Check for position bias, self-preference and agreement with blinded human examples; another model is not automatically independent evidence.

Separate infrastructure failure, invalid task or grader, missing observation, and implementation failure. Report reliability including infrastructure failures, and capability results on valid runs; do not silently drop difficult cases. Delayed regressions and reverted PRs update the delivery outcome with attribution uncertainty. Run cheap contract checks for every task; budget model judging for sampled, ambiguous or failed work.

Use MAST's distinction between system design, inter-agent alignment and verification failures as a starting diagnostic taxonomy [21]. These labels identify hypotheses to investigate, not a proof that a particular prompt or agent caused the outcome.

18 Datasets and independent evaluation environments

Build a private evaluation set from real issues, regressions and representative engineering tasks. Each case includes a repository snapshot, clear task statement, input fixtures, required environment, explicit acceptance criteria and an independently maintained grader. The candidate never receives its hidden reference patch or private acceptance assertions.

Dataset design

1. Start with 30 to 50 well-specified cases spanning bug fixes, APIs, schema changes, frontend work and integration. This is an initial curation target, not sufficient evidence for every automatic promotion.
2. Group cases by original issue and patch lineage before splitting. Near duplicates, children of the same delivery and repeated trials stay together.
3. Maintain development cases for diagnosis, a validation set for candidate selection, and a protected certification set for limited final checks.
4. Version case inputs, dataset membership and graders separately. Add recent private tasks prospectively to reduce repeated holdout adaptation.

Execution harness

Run candidate and incumbent in fresh isolated environments from the same base commit, dependency lockfiles, tools and fixtures. Reset databases and services between trials. Keep unrelated network effects and live production writes out of evaluation; model calls still use the explicitly selected authorised profiles.

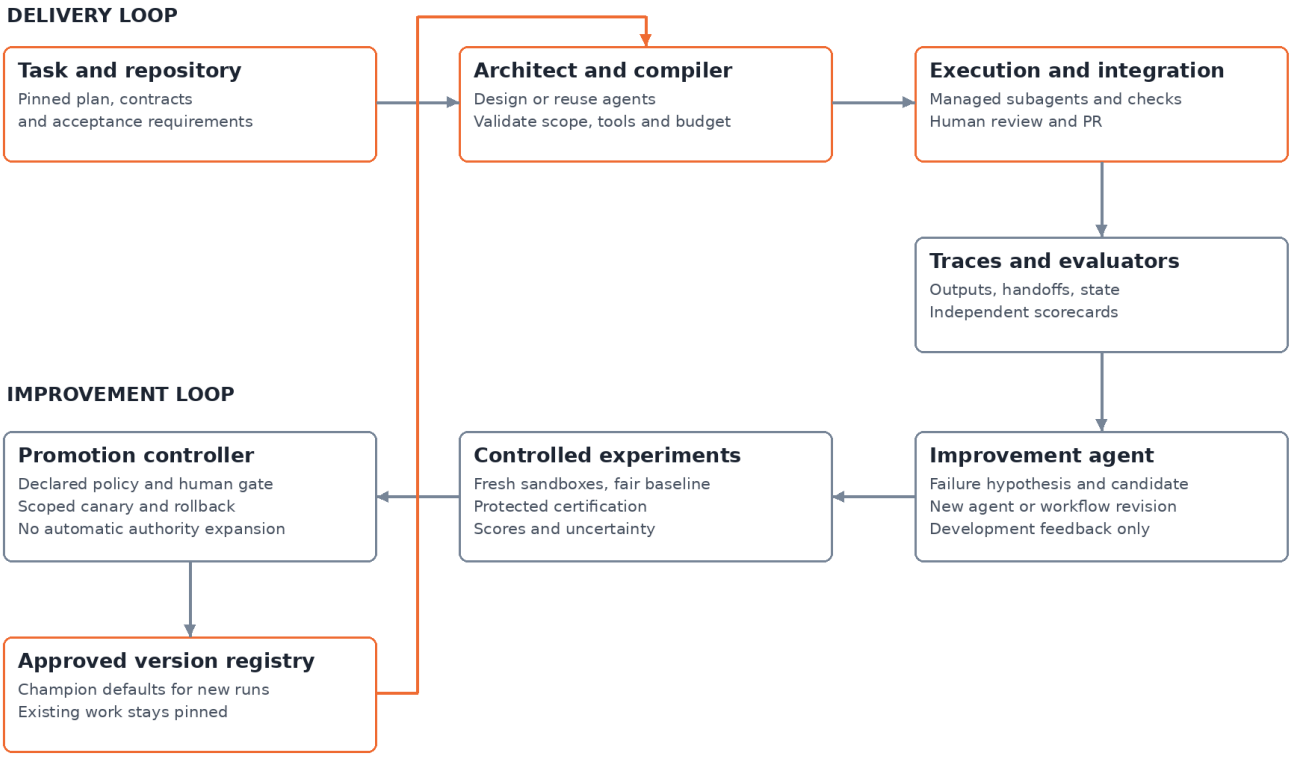
Run hidden acceptance tests in a separate evaluator environment where candidate code cannot replace the grader or alter its result files. Check that a known bad or original revision fails the relevant assertion and an accepted reference implementation passes. Evaluate behaviour rather than requiring an identical patch.

Evaluator integrity

Keep grader configuration, protected dataset access and promotion criteria outside worker and optimiser write access. Candidate-generated tests can become proposed cases after review; they cannot immediately define the metric that certifies the same candidate. Calibrate grader changes with known good and bad outputs and re-score baselines when a grader changes.

Stored-trace re-scoring is useful for diagnosis and judge calibration [29]. It cannot show that a newly proposed workflow will choose better tools or produce a correct changed repository. That requires a fresh execution. Record partial observability and keep unsupported conclusions out of promotion decisions.

19 The measured improvement loop



Two connected loops. The delivery loop solves the current task. The improvement loop tests reusable changes before they affect future deliveries.

The observer and evaluator are services with deterministic collection and grading paths; model judges assist only where useful. The improvement agent reads sanitised evidence and proposes candidate changes. A separate controller runs comparisons, applies the declared promotion policy and records why a version was accepted or rejected.

A local repair can adapt the current task inside its approved envelope. A reusable change is a new immutable version. Existing runs retain their pinned definitions; new runs use a promoted default only within its approved task family and project scope.

Continuous improvement means this cycle can keep generating and testing hypotheses. It does not guarantee every cycle produces a gain or that the system can establish quality when the evaluation evidence is weak.

20 Improvement agent and the search space

Give the improvement agent read access to eligible traces, evaluation summaries, development cases and prior candidate results. Its write access is limited to a candidate branch or proposal store. It can request bounded experiments, not edit the active evaluator, protected cases, employee grants or deployment policy.

Consume delivery and evaluation completion events continuously. Run lightweight scoring on every result, and open an analysis job after a repeated failure, an explicit team request or a configured evidence window such as 20 new valid outcomes. This threshold triggers diagnosis, not permission to promote. Keep experimental work behind delivery execution in the queue.

What it may propose

- Agent instructions, input and output schemas, skill bundles and context selection rules.
- When to keep one worker, split work, add a reviewer, merge two roles or change dependency ordering.
- Assignments to permitted harness and model combinations, with comparable resource budgets.
- Repair and stopping criteria inside the approved limits; repository bootstrap or tool improvements through ordinary reviewed code changes.

Proposal procedure

1. Identify a repeated failure or cost pattern and cite the underlying runs and artifacts.
2. State a falsifiable cause hypothesis, expected benefit, affected task family and possible regression.
3. Produce a small versioned diff and select an unchanged evaluation contract.
4. Run the cheap screening tier, then request a budgeted candidate-versus-incumbent experiment.
5. Return the measured result and uncertainty, including negative results; do not just write a new prompt and declare improvement.

Research to use

ADAS demonstrates searching agent designs, AFlow searches workflow structure, and GEPA uses execution feedback to propose and compare textual changes [17, 18, 19]. These motivate a measured search loop. Their benchmark results do not establish gains on our private repositories.

Start with one proposer and a few targeted candidates. Add GEPA as a local optimisation helper when repeated experiments justify it; its general evaluator interface can work with our harness [20]. We do not need to move the production agents into DSPy, LangGraph or another runtime to adopt the optimisation method.

Use typed workflow edits first. New executable tools or platform code go through the existing code review and integration process. Pin a champion and retain candidate failures so that the optimiser does not keep retrying a rejected design without new evidence.

21 Controlled experiments and promotion

Fair comparisons

Compare the candidate with the current champion on the same case snapshots and task mix. Include a strong single-agent and a static-team baseline when testing whether dynamic delegation adds value. Control model access, available tools, runtime and total usage where observable. Randomise run order and repeat promising comparisons because native model calls are stochastic.

Compute paired differences by case; bootstrap or otherwise report uncertainty at the original issue or delivery cluster, not by treating every child span as an independent sample. Report important task-family regressions, human intervention and integration costs. Matched settings do not make runs exactly reproducible, and unavailable usage data prevents claims of equal token budgets.

Evaluation budget

Use a ladder: schema and policy checks, a small smoke set, development comparison, protected certification, then a scoped prospective canary. Begin with at most 2 candidates per experiment and one experiment worker. Configure run, wall-time and usage ceilings. A 20-case comparison with 3 trials for both candidate and baseline is 120 task runs; approval of the experiment budget is separate from the optimiser's proposal.

Promotion rules

1. Reject any candidate that violates required policy, fails a critical acceptance case, lacks necessary trace coverage or changes its own grader.
2. Require a predeclared useful gain or non-inferiority criterion with resource benefit. Keep the champion when the evidence is inconclusive.
3. Initially present the version diff, cases, results and known regressions to the delivery or platform owner for promotion.
4. Later allow automatic promotion only for preauthorised low-risk change classes that meet fixed evidence and rollback rules.
5. Assign the candidate to a small randomised cohort of new deliveries. Route each delivery to one champion; do not duplicate real writes for live A/B tests.
6. Roll back the default version on a verified regression, preserve evidence and stop expansion. In-flight work stays pinned unless explicitly cancelled or handed over.

Protected certification is rate-limited. Repeatedly revealing detailed holdout failures turns it into development data. Refresh cases and use prospective evaluation; allow the optimiser to receive only the certification outcome needed for a promotion decision.

22 Self hosted observability and evaluation stack

Use OpenTelemetry for instrumentation and self-hosted MLflow as the first trace and experiment backend. Build our product-specific agent graph, grants, workflow state and promotion logic in the control plane. The telemetry store is a replaceable projection, not another orchestration authority.

Component	Choice	Ownership
Execution record	Existing PostgreSQL events and private artifacts.	Canonical tasks, decisions, leases, definitions and evidence refs.
Instrumentation	OpenTelemetry SDK and authenticated OTLP export.	Custom adapters capture native CLI events; pin schema version.
Trace and experiment UI	Self-hosted MLflow.	Separate database credentials and artifact prefix; internal access controls.
Evaluation worker	Pinned Python container with repository-specific graders.	Fresh sandboxes; custom model judges via approved profiles.
Improvement	Normal managed agent; optional local GEPA.	Candidate proposals and bounded experiments.
Alternative backend	Langfuse if its trace and dataset UI is preferable.	One backend at a time; no requirement to deploy both.

MLflow accepts OTLP traces and can use a SQL backend, so it can reuse our existing PostgreSQL infrastructure with a separate database and role [27, 28]. The evaluation worker records our custom scores and artifacts. Core tests do not require hosted judges or a Databricks account.

Langfuse supplies an agent graph interface [30]. Its documented self-hosting stack adds ClickHouse and Redis or Valkey alongside PostgreSQL and object storage, with some governance add-ons licensed separately [31]. Choose it when those capabilities justify the additional operations.

Self-hosting removes a mandatory external observability SaaS dependency. Native Claude and Codex model access still follows their configured providers; judge or improvement calls also consume authorised account capacity. Add no new hosted model dependency implicitly. If a proposed optimisation helper requires a different credential path, make that an explicit service profile or use the native agent proposer.

Graph views inferred from timing and nesting are useful for navigation. Our dependency and handoff graph must come from explicit events and IDs; a visual inference is not authoritative causal evidence.

23 Versioned records and new service contracts

Record	Required contents
AgentSpec revision	Instructions, capabilities, context, output and eval contract; immutable hash.
Agent instance	Definition, parent, task, profile, native reference, status and limits.
Workflow revision	Typed graph, contracts, limits and input versions.
Telemetry coverage	Observed event classes, gaps and adapter version.
Eval contract and result	Scorer version, criterion, evidence, confidence and validity.
Case and dataset revision	Repository/environment snapshot, split membership and private graders.
Experiment and trial	Candidate, champion, case, repeated trial, budgets and paired results.
Change proposal	Hypothesis, supporting failures, diff, affected scope and experiment request.
Promotion and rollback	Policy decision, version assignment, canary cohort and observed outcome.

Platform tool surface

```
propose_agent(spec, source_task)
compile_agent(spec_revision, policy_revision)
spawn_child(compiled_spec, task, authorised_profile)
report_handoff(child_id, output_artifact)
submit_eval_result(contract, evidence, validity)
propose_change(champion, diff, hypothesis)
request_experiment(candidate, dataset, budget)
request_promotion(experiment, scope)
```

Each mutation is idempotent, version-checked and permission-checked. A successful proposed definition is not a spawned instance. A successful experiment request is not approval to use an account. A positive evaluation result is not an automatic promotion unless the independently configured policy allows it.

Resolve all referenced skills, tools, models, context packs and policies before launching an instance. Store their versions in the run manifest. Never mutate a running definition in place. Distinguish a task-level continuation from a new agent revision or a change to the project default.

The optimiser reads the sanctioned development feedback view. Protected case artifacts, grader secrets and promotion credentials are served by separate roles. Keep approval and promotion records outside the optimiser's writable context even if it can edit its candidate workspace.

24 Persistent data and service interfaces

Entity	Core fields and constraint
User and membership	Company identity, role, project grants; revocable.
Runner	Owner, device key, capabilities, health, drain state.
Execution profile	Owner, account reference, agent version, runner, grants.
Project and repository	Remote, bootstrap profile, checks, permitted runners.
Delivery and plan revision	Immutable approved plan, requirement map, base SHAs.
Task and dependency	Objective, ownership, acceptance, predecessor references.
Attempt and lease	Task, generation, profile, native session, active slot.
Approval and decision	Bound input hash, authorised deciders, result.
Event and command	Unique source ID or idempotency key; ordering metadata.
Artifact and check result	Private object reference, SHA, type, provenance.
Integration and PR	Included task commits, integration SHA, CI, PR identity.
Usage and audit entry	Reported usage, estimates, actor, action, scope.

API boundaries

```
POST /deliveries                create with idempotency key
POST /deliveries/:id/plans       propose a new revision
POST /plans/:id/approve          require expected revision
POST /tasks/:id/attempts         start or retry under policy
POST /attempts/:id/messages      ordered, idempotent input
POST /attempts/:id/interrupt     scoped control command
POST /approvals/:id/resolve      compare-and-set decision
POST /runners/enrol              single-use invitation
GET  /projects/:id/events        SSE with resume cursor
POST /integrations/:id/prepare-pr exact accepted SHA
```

All endpoints enforce project permissions and resource ownership on the server. Reads and live subscriptions use the same access checks. Terminal, file, and preview routes must not become alternative unrestricted access paths. Never accept an arbitrary filesystem path from the browser.

Extend the schema with the immutable definitions, instances, coverage, evaluations, experiments and promotion records described earlier. Use a transactional outbox for dispatch, evaluation requests, telemetry projection and notifications. Apply events to normal database state in the same transaction as the durable event record. This is an append-only audit and replay stream, not a requirement to rebuild the entire product as an event-sourced system.

25 Runner protocol and workflow configuration

Event envelope

```
{
  "schemaVersion": 1,
  "eventId": "runner-local-unique-id",
  "runnerId": "runner_a",
  "attemptId": "attempt_17",
  "generation": 3,
  "sequence": 84,
  "type": "approval.requested",
  "occurredAt": "2026-11-02T12:00:00Z",
  "payload": { "nativeRequestRef": "req_9" }
}
```

Deduplicate on runner and event ID; enforce increasing local sequence per attempt. Arrival time is not causal ordering across runners. Acknowledge only after durable persistence. On reconnect, replay events from the last acknowledgement. If native delivery of a command is uncertain, inspect the session or ask for resolution rather than resending it blindly. Store high-volume text in chunks and large artifacts in object storage.

Example delivery configuration

```
delivery: project-audit-log
base: <resolved-repository-commit>
plan_approval: required
limits:
  active_tasks: 3
  attempts_per_task: 2
  child_tasks_per_parent: 3
steps:
  - id: contract
    kind: agent
    profile: owner-a-claude
    output: approved-contract
  - id: implementation
    kind: parallel_tasks
    depends_on: [contract]
    tasks: [storage-api, console-ui, sdk]
  - id: integrate
    kind: integration
    depends_on: [implementation]
    checks: repository-integration
  - id: release-review
    kind: human_gate
    scope: integration-sha
```

This is a proposed platform schema, not a configuration accepted by an existing tool. Resolve names into validated internal IDs when approving the plan. Limit step types initially to agent, check, human_gate, parallel_tasks, and integration. Render the stored graph in the UI, so configuration and execution cannot disagree.

The coordinator receives scoped tools to propose tasks, inspect related status, ask a question, and send bounded messages. It has no tool that creates account grants, widens budgets, or declares its own output accepted.

26 Recovery and operational behaviour

Failure	Required response
Browser disconnects	Execution continues; reconnect with event cursor and current decision state.
Control service restarts	Recover leases and outbox; reconcile runners before starting replacements.
Runner loses connection	Journal locally; stop new dispatch; park or interrupt after lease grace.
Native process dies	Preserve branch and logs; mark interrupted; offer a supported resume.
Login expires	Set auth_required; owner signs in; no silent account switch.
Quota exhausted	Set quota_wait; show reported reset or unknown; preserve work.
Approval expires	Reject or interrupt according to adapter; a stale click has no effect.
Duplicate dispatch arrives	Return the existing attempt; never launch a second process.
Old runner reconnects	Reject stale generation; quarantine branch output until reconciled.
GitHub webhook repeats	Deduplicate delivery ID; verify current PR and commit state.
Disk pressure	Stop new preparation; preserve unmerged work; clean eligible artifacts.

Metrics and support tools

Track queue wait, preparation duration, active and waiting workers, account health, approval age, repair loops, integration failures, memory and disk use, and delivery completion. Trace each operation through delivery, task, attempt, and runner identifiers. Never label an estimated subscription charge as an invoice or a missing quota reading as zero.

Provide a diagnostic bundle containing redacted environment metadata, adapter version, process state, command acknowledgements, and recent events. Make bundle creation explicit and keep it private to the project. Redact credentials before durable log storage, not only when displaying the page.

Retention and recovery

Start with 30 days of full operational logs, 90 days of delivery evidence, and longer retention of compact decisions and merge records as a configurable internal policy. Preserve unmerged branches and pinned artifacts independently of routine cleanup. Cleanup is a resumable job with an eligibility check, not an agent deciding what to delete.

Back up PostgreSQL daily and protect object storage versions. Keep runner credential recovery under the owner's native login process. Restore the control plane in a test environment during the pilot and prove that it does not duplicate active work. Mark unreachable external side effects as uncertain until reconciled.

Roll back a merged change through a reviewed revert PR and its checks. Restoring a workspace checkpoint does not undo a database migration, deployed service, or external side effect. Record any required compensation or migration recovery in the delivery before execution.

27 Deployment capacity and cost controls

Separate the control service from build workers. The control plane needs dependable storage and network availability; parallel builds and browser tests belong on the runners. The following are initial sizing estimates to validate against our repositories, not throughput benchmarks.

Component	Pilot starting point	Scaling trigger
Control plane	4 vCPU, 8 to 16 GB RAM, PostgreSQL, private artifact storage.	Event lag, query latency, or sustained API pressure.
Employee runner	4 to 8 vCPU, 16 to 32 GB RAM, SSD workspace disk.	Measured build or test memory and queue wait.
First fleet	3 employee runner boundaries; 6 active attempts total.	Successful pilot and available account capacity.
macOS runner	Existing or dedicated Mac mini sized for actual builds.	Required Xcode or macOS workload.

Hosted-model CLIs do not need a worker GPU. CPU, memory, disk, browser processes, test databases, and native toolchains determine local capacity. Pause dispatch before memory pressure becomes a host failure. Reserve capacity for integration and review so coding workers cannot starve completion.

Installation

1. Create the private GitHub repository and OIDC application; configure the first allowed repositories.
2. Deploy the control service, database, object storage, TLS endpoint, and backups in the company's existing infrastructure.
3. Enrol three employee runners and pin one compatible version of each CLI.
4. Have each employee sign in and approve the projects and colleagues allowed to use their execution profiles.
5. Register repository bootstrap and validation profiles; verify private submodule access.
6. Run a fake-runner scenario, then a real low-risk feature through integration and human PR review.

Usage and budget policy

Track native reported tokens and account quota when supported. API profiles can have monetary ceilings; subscription profiles primarily need concurrency, runtime, and quota controls. If cost cannot be measured reliably, enforce time and turn limits and label any estimate clearly. Child tasks and competing attempts share the parent's remaining budget.

Provide account availability windows and manual drain controls. Draining prevents new assignments but lets authorised current work finish. Reassignment to a different account is explicit and records the new owner and context handoff; it is not automatic quota rotation.

28 Mubit context and learning

Use Mubit for approved repository context, validated lessons and retrieval of relevant prior experiment findings. Dynamic agent design, evaluation and improvement are core platform features and do not depend on this optional memory service. The orchestration database remains authoritative for live tasks, approvals, leases, and repository state. Memory helps agents find relevant evidence; it does not decide permissions or replace Git.

Useful memory records

- Repository conventions approved by the team, with applicable paths and toolchain versions.
- Validated fixes linked to the original issue, relevant commits, test results, and whether the fix remains current.
- Past architecture decisions with their owner, rationale, date, and superseding decision.
- Observed agent performance on categories of real tasks, including integration repairs and reviewer effort.

Read and write flow

At task preparation, retrieve a small scoped context pack for the project, pinned commit, relevant files, and task type. Include evidence links and freshness information. The worker can request more evidence through a project-scoped tool. If retrieval fails, continue with repository instructions and the approved plan.

A dedicated ingestion job writes a candidate memory only after checks and review, or records a clearly labelled failed experiment for diagnosis. Hypotheses and failed candidates never become active instructions merely because they were stored. Engineers can correct, supersede, archive, or remove a record. A stale convention is superseded when a later reviewed decision changes it. Preserve the lineage from a memory record to its source commits and decisions.

Agent selection

Begin with explicit assignments and simple observed statistics. Later suggest the agent and model that previously completed similar tasks with the least combined execution, review, and repair effort. Display sample size and uncertainty. Do not automatically change the team's routing based on one successful task or unreviewed model self-scores.

Data boundaries

Keep memories scoped to the company project and its permitted readers. Avoid copying raw credentials, unrelated personal session history, or entire transcripts into memory. Support removal of source material and derived context packs. Any fine-tuning on company code or traces is a separate opt-in project with its own data and evaluation plan.

Mubit integration is optional in the first release. The first useful checkpoint is repository conventions and reviewed decisions; outcome-based routing comes after enough comparable deliveries exist. This section defines proposed integration behaviour rather than assuming a particular current Mubit SDK contract.

29 Worked delivery example

Example goal: add project-level audit-log viewing to the Mubit console and expose the same records through the API and SDKs. This is an illustrative delivery that exercises the orchestration design, not a statement about the current implementation.

Task	Assignment	Dependency and output
T1 Contract	Employee A, Claude Code	Pinned main; event fields, API contract, permissions and acceptance cases.
T2 Storage and API	Employee B, Codex	Approved T1 contract; migration, storage writes, filtering and API checks.
T3 Console	Employee C, OpenCode	Approved T1 contract; UI against agreed fixtures; isolated preview.
T4 SDK surface	Available authorised profile	Approved T1 contract; Python and JS changes with compatibility checks.
T5 Integration	Integration service plus repair agent if needed	Accepted T2 to T4; combined SHA with end-to-end checks.
T6 Review and PR	Designated reviewer and delivery owner	Validated T5; findings resolved and final human merge decision.

The coordinator proposes these tasks, explains why T2 to T4 can overlap, and flags shared schema or generated files. The delivery owner approves the contract and assignments before the parallel stage starts. Each worker receives the same contract revision and only the context relevant to its task.

Suppose T3 needs a decision on whether deleted users remain identifiable in the audit view. It opens a question tied to the requirement and contract revision. An authorised engineer answers in the decision inbox. The coordinator assesses the impact, creates a revised contract if necessary, and pauses only tasks whose inputs changed.

Suppose the Codex profile reaches a quota limit during T2. T2 becomes quota_wait, while independent T3 work continues. The owner can wait for a verified reset or deliberately reassign the task with a recorded handoff. The system preserves T2's branch and does not silently switch to another employee's subscription.

After T2 to T4 pass review, the integration service combines them and runs API-to-console tests on the integrated SHA. If a shared type is incompatible, a repair task receives the actual diff and failure. The final PR includes the implementation, checks, migration ordering, and the decision about deleted users. The human reviewer sees one coherent change set.

30 Worked agent design and improvement example

Use the audit-log delivery to demonstrate both loops. The task architect first designs a migration author, an API implementer, a console implementer and a read-only compatibility reviewer. Each gets a different input pack and acceptance contract, then binds to an already authorised employee CLI profile.

What the designer changes

The migration author receives schema history, database fixtures and forward-compatibility checks. The API implementer receives the accepted contract and storage interface. The console agent receives fixtures and the UI acceptance cases. The reviewer receives the combined diff and public contract; private acceptance tests remain in the evaluator environment.

What the observer records

The product graph shows why each specialist was created, which AgentSpec revision it used, its handoffs, visible tool calls, waiting periods and the actual integration result. Native child activity appears only at the detail level supported by that adapter. The scorecard separates local correctness from shared-interface and integration failures.

What an improvement experiment tests

A repeated pattern of integration repairs can motivate a candidate workflow that pins a reviewed contract before parallel implementation, rather than letting each implementation agent edit shared types independently. The hypothesis is reduced integration repair without lower completion or excessive reviewer work. This is a proposed experiment, not a measured result.

Element	Experiment design
Champion	Current approved team structure and context policy.
Candidate	Contract owner first, narrower implementation scopes and explicit reviewer handoff.
Controlled inputs	Same issue snapshots, available models, tool versions, fixtures and declared budgets.
Main evidence	Hidden acceptance, combined build, integration conflicts and repair time.
Additional costs	Agent usage, reviewer time, total latency and experiment expense.
Decision	Promote within this task family only if the declared gate passes; otherwise keep the champion.

The improvement agent can propose removing a specialist as well as adding one. A smaller workflow that meets the same acceptance contract with less review or repair can be the better candidate. Preserve both successes and failed experiments so later designs have evidence to build on.

31 Delivery roadmap and staffing

Use three workstreams: control-plane state, definitions and scheduling; runners, adapters and event coverage; evaluation harnesses, experiments and the workbench. Shankha owns product priorities and the initial promotion policy. With two engineers, sequence the evaluation work after the first traceable delivery slice without omitting instrumentation from that slice.

Phase	Effort	Concrete exit gate
0 Capability proof	7 to 10 engineer-days	Three CLI profiles; custom roles; child visibility; approval and coverage limits documented.
1 Traceable vertical slice	13 to 18 engineer-days	One real task with correlated lifecycle, check and evaluation records.
2 Designed team	18 to 24 engineer-days	Typed AgentSpec compiler, broker, owner grants and shared child graph.
3 Parallel verified delivery	20 to 26 engineer-days	Dependency scheduling, independent graders and integrated PR evidence.
4 Reliable pilot	14 to 18 engineer-days	Recovery, partitions, dataset isolation, backup and restore tests.
5 Controlled improvement	21 to 30 engineer-days	Candidate proposals, paired experiments, certification, canary and rollback.
6 Custom additions	7 to 14 engineer-days	macOS and multi-repo acceptance; optional Mubit and optimiser extensions.

Total estimate: 100 to 140 engineer-days, roughly 35 to 55 more than the earlier 65 to 85 day scope. Plan 9 to 13 calendar weeks with three experienced engineers, or 13 to 19 with two. These ranges include engineering work, not a promise that enough representative data or statistically clear gains will exist by a fixed date.

First five working days

1. Define AgentSpec, the broker policy envelope and the telemetry event contract.
2. Probe each installed CLI for custom role configuration, child lifecycle, visible tools, questions, usage and resume support.
3. Instrument a real parent and child flow and show a supported team decision without exporting credentials.
4. Create five private task fixtures with independent deterministic checks and known good and bad results.
5. Compare one managed specialist with one unsplit baseline; document gaps and pin the supported versions.

Deliver useful milestones continuously. Instrumentation and evaluation start in the first real slice. Automatic reusable promotion waits for the protected experiment and rollback path; candidate generation and human-reviewed experiments can be useful before then.

32 Implementation backlog

Treat the following as independently reviewable tickets. Each ticket produces code, a working scenario, and evidence appropriate to its failure modes. Group related UI and API changes into vertical slices rather than waiting for all back-end work to finish.

ID	Deliverable	Depends on
01	Monorepo, contracts, migrations, CI and fake runner.	None
02	OIDC login, project membership and authorisation checks.	01
03	Runner enrolment, device identity and heartbeat.	01, 02
04	Runner process supervisor, journal and reconnect.	03
05	Native CLI probes and pinned capability registry.	04
06	Claude adapter and supported decision bridge.	05
07	Codex stdio adapter and request/reply bridge.	05
08	OpenCode versioned adapter and local API bridge.	05
09	Owned execution profiles and delegation grants.	02, 05
10	Repository registration, bootstrap and submodules.	04, 09
11	Task workspace lifecycle, ports and check capture.	10
12	Live task workbench and terminal controller lease.	04, 11
13	Decision inbox, atomic answers and stale-request handling.	06 to 09, 12
14	Plan revisions, requirements, task graph validation.	02, 11
15	Coordinator tools and task proposal review.	13, 14
16	Scheduler, fair limits, leases and outbox dispatch.	04, 09, 14
17	Parallel tasks, handoffs and bounded child proposals.	15, 16
18	Independent review and result acceptance.	11, 13, 17
19	Integration lease, dependency order and repair tasks.	17, 18
20	GitHub App, PR creation, CI and review feedback.	18, 19
21	Quota waits, auth recovery and explicit reassignment.	06 to 09, 16
22	Crash and partition reconciliation tests.	04, 16, 19, 21
23	Metrics, retention, backups and restore drill.	20, 22
24	Pilot deployment, onboarding and runbook.	13, 20 to 23
25	macOS, multi-repo and optional Mubit context slices.	24

Extend these original platform tickets with the agent, telemetry and evaluation backlog next. Instrument contracts during ticket 01 and adapters during 06 to 08; do not wait for a finished orchestration product to add observability. Native compatibility, independent grading and integration evidence are critical path work.

33 Agent evaluation and improvement backlog

ID	Deliverable	Depends on
26	AgentSpec schema, immutable registry and compiler.	01, 02, 05
27	Trace schema, durable event projection and OTLP export.	01, 03, 04
28	Delegation broker, child grants, budgets and cancellation.	09, 16, 26
29	Native child adapters and coverage capability tests.	06 to 08, 27, 28
30	Shared agent graph, handoff inspection and version views.	12, 26 to 29
31	Evaluation contracts, deterministic scorers and known-result calibration.	11, 18, 27
32	Private case store, versioned splits and independent sandboxes.	10, 11, 31
33	MLflow integration, scorecards and experiment evidence.	27, 31, 32
34	Calibrated judge adapter and uncertainty handling.	06 to 09, 31, 32
35	Improvement proposer, diagnosis and candidate diffs.	26, 30 to 34
36	Paired experiment scheduler and protected certification.	16, 32 to 35
37	Promotion policy, cohort assignments and rollback.	13, 20, 36
38	Gaming, lineage leakage, uncontrolled child and evaluator tests.	28 to 37
39	Full pilot with a rejected candidate and a controlled rollout.	24, 30 to 38

Implementation sequencing

The IDs extend the earlier backlog; they do not imply implementing all original tickets before 26. Start 26 and 27 alongside the base contracts, add 28 during parallel scheduling, and complete 31 and 32 before autonomous improvement experiments. Use the phase exit gates to decide when the next capability can run on real work.

Additional acceptance scenarios

- The architect creates an unfamiliar specialist from a real task and its definition compiles without manual role coding.
- Every managed child appears before execution; native gaps are visible and do not bypass the global cap.
- A candidate cannot modify its private grader or read the protected case answers.
- The optimiser proposes an observable change; a controlled experiment rejects a deliberately worse version.
- A promotion affects only eligible new deliveries, and rollback restores the previous default without rewriting old evidence.

34 Verification plan and operational acceptance

Test the orchestration properties that can lose work, mix identities, or produce an invalid combined change. Use fake agents for deterministic failures and small real-agent canaries for compatibility. Avoid broad expensive model runs when a protocol or domain test can prove the property.

Scenario	Pass condition
Account isolation	A job cannot read another employee VM's credentials; selected account identity matches the profile.
Permission separation	A viewer cannot prompt, approve, use a profile, or send terminal input without the relevant grant.
Parallel branches	Three agents change independent modules without sharing a checkout; combined checks pass.
Graph validity	Cycles, missing inputs, unavailable profiles, and unsupported actions are rejected before dispatch.
Decision race	Two simultaneous answers produce exactly one recorded authoritative resolution.
Stale approval	A changed command, native request, plan revision, or SHA cannot consume the old approval.
Network partition	The old runner is fenced from publishing after replacement; uncertainty blocks unsafe duplicate writes.
Duplicate delivery	Repeated dispatch, events, messages and webhooks do not duplicate processes or side effects.
Restart and resume	API and runner restarts preserve task state, local work, pending input, and event order.
Quota and auth	Exhaustion or expired login preserves work and never silently switches employees.
Integration conflict	A repair receives both branches; checks run on the actual repaired integration SHA.
Private submodules	Preparation fetches required content or stops with a clear actionable failure.
Restore	A restored control service reconciles existing runners without starting duplicate work.

Operational targets

Use initial service targets of under 2 seconds for connected-runner event visibility and under 5 seconds for a supported decision to reach the web inbox, at pilot load. These are proposed acceptance targets, not claimed performance. Measure them with replayed transcripts and active real-agent sessions.

Target zero unauthorised profile use and zero duplicate authoritative integration writes in the fault suite. Track task retry rate, stale decision count, integration repair time, and human review effort. Final approval requires a real dynamically designed delivery, a restore drill and a controlled improvement experiment. Validate known good and bad candidates, private grader isolation, delegation caps and rollback; passing unit tests alone is insufficient.

35 Rollout and decisions to make during the pilot

Staged adoption

1. Run the platform on a low-risk repository with three employee volunteers. Keep the usual branch protections and human PR review.
2. Complete 5 small deliveries that exercise different languages or build paths. Capture review and repair effort against the team's existing approach.
3. Complete 3 parallel deliveries with dependencies, at least one contract change, and an integration failure that the platform repairs transparently.
4. Enable unattended bounded work only for workflows whose profile, checks, and recovery paths have already passed the pilot.
5. Expand the runner fleet and repositories gradually. Evaluate macOS and multiple repositories as separate acceptance milestones.

Operational runbook

Before launch, check runner reachability, native account health, repository access, available disk, and the validation profile. During a delivery, use the decision inbox for unresolved work and the fleet view for infrastructure failures. Before merge, inspect the integrated diff, required checks, and actual GitHub reviewer state.

When work stalls, identify whether it is waiting on a dependency, human decision, authentication, quota, infrastructure, or a failed check. Preserve the existing attempt before choosing resume, repair, or a new assignment. When shutting down a runner, drain it and confirm process and branch preservation before host maintenance.

Highest impact uncertainties

Question	How to resolve it
Can each installed CLI surface the needed decision types?	Phase 0 real-account tests; terminal fallback for unsupported controls.
How much concurrency can our repositories sustain?	Measure build memory, test services, ports and integration throughput.
How much autonomy does the team actually want?	Start with approved plans and explicit merge; adjust per workflow after evidence.
Which agent is best for each type of work?	Collect reviewed outcome data before introducing automatic routing.
Does a shared platform save total engineering effort?	Compare delivery, review and repair time over real tasks.

Proceed with the design defaults in this plan. Use pilot evidence to revise supported agent versions, account isolation, concurrency and improvement policy. Default reusable changes to human-reviewed promotion; later permit automatic promotion only for preauthorised change classes with enough evidence. Keep those decisions visible in the product so that later changes are reviewable.

36 Event intake and scoped work authority

Adopt the useful operating patterns of the DeepSeek dsh-superagent prototype: event intake, scoped work authority, persistent follow-ups and low-noise supervision. Its README calls it a research prototype for one repository built on DeepSeek Harness [35]. Keep our independent control plane, native CLI adapters and employee-owned profiles.

Two different kinds of evaluation

The prototype calls its intake decision Eval [36, 40]. Name ours AdmissionDecision to distinguish it from the quality evaluations that score agent outputs and completed deliveries. A message can be valid context without authorising new work.

1. A Source adapter records a canonical immutable event, verified actor metadata, conversation reference and source cursor. Persist before advancing the cursor; deduplicate by source and native event ID.
2. A deterministic gate checks project membership, source permissions, explicit request status and the current policy revision. The classifier can interpret intent only inside that envelope.
3. Route an in-scope follow-up to its existing task. An explicit task or PR reference can bind it; semantic similarity alone supplies context.
4. New scope becomes a proposed delivery or task. The architect decomposes it only after admission and the applicable plan approval.

Proactive intake mode	Behaviour
Off	No inferred work candidates.
Observe	Record the decision and proposed scope; launch no task and send no acknowledgement.
Auto	Dispatch only a matched, preauthorised rule within its grant and budget.

Work mandate

Record origin_event_id, requester, project, scope constraints, policy revision, rule IDs, account-grant reference, budget and expiry. Child mandates must be subsets. Reject stale admission results after a policy change. Revalidate at dispatch and resume; hard grant revocation blocks new operations immediately. The prototype treats some live policy changes differently [36]; retain our existing revocation guarantees.

Manual plan submission remains the first intake path. GitHub issue, review and CI intake is an optional extension. Observation still consumes classifier capacity and must use an authorised profile and budget.

37 Follow ups and reliable message handling

The prototype keeps work ownership separate from conversations and executor sessions [37]. Extend our existing tasks and attempts with conversation subscriptions and explicit follow-up handling. Avoid introducing a second Ticket object for the same work.

Conversation routing

A task can follow several issue or PR threads, and a thread can supply context to several tasks. Subscribing does not expand scope. For each actionable follow-up, record one designated handler; other subscribers receive context only. The handler resolves it within its mandate or returns it with a reason for new admission.

An approved human follow-up can resume the existing task after current permission checks. Bot reviews and CI events may wake eligible active work, but cannot override a human stop or create a new mandate. Out-of-scope requests and ambiguous references enter the decision inbox.

Record what happened to each input

Record	Meaning
Received	The platform persisted the source event.
Delivered	The adapter acknowledged delivery to the native session.
Read	The adapter observed the input entering session history; otherwise unknown.
Handled	The worker returned a structured acknowledgement, answer or hand-back.
Settled	The result or rerouting decision is durably recorded.

Do not infer read or handled status from a successful transport write. Preserve unread and unsettled inputs across attempt replacement; reconcile native IDs before replay. These distinctions appear in the prototype executor hooks and outbox [41, 42]. Native CLIs without read acknowledgements expose a coverage gap.

Structured reports and outbound receipts

Worker reports carry status, checked commit, artifact references, handled input IDs and an optional human_action with one concrete question, intended decider and blocking scope. The router forwards the question with its evidence. Independent work can continue.

Use the existing transactional outbox for acknowledgements and results. Store receipt ID, task, destination, purpose and pending, sent, dropped or uncertain status. Reconcile uncertain sends using provider IDs before retrying. Per-task conversation ordering must not block unrelated work; do not claim exactly-once delivery across an external service boundary.

38 Prompt upgrades and human attention

Adopt the prototype prompt editor model: separate shared instructions, role instructions, team conventions, delivery rules and skills, and retain the built-in default on which an override was based [38]. This makes runtime upgrades reviewable without losing team customisations.

Versioned prompt bundles

Add PromptBundle revisions to the existing agent registry. Each bundle records component hashes, default-base hashes, team overrides, required template fields and skill versions. The editor shows the previous default, new default, custom text and affected evaluation cases. Validate size, known component names and placeholders; preserve a conflicting edit as a draft.

Unlike the prototype live prompt reload, keep each attempt pinned to its compiled bundle. A prompt upgrade is an improvement candidate and affects future attempts only after its evaluation and promotion rules pass. A manual steering input is a separate recorded intervention. Immediate access revocations remain deterministic policy checks, independent of prompt versions.

A workbench organised around decisions

Show questions and required reviews first, actionable operational problems second, and history and detailed traces on demand. Use plain labels that identify who can unblock work and what they need to decide. Routine tool activity stays in the trace. A task sends one acknowledgement and then a useful result, decision request or failure notice; teams can configure this communication policy.

Measure	What it helps evaluate
Human intervention minutes	Time spent answering, reviewing and repairing a delivery.
Questions and notifications	Whether communication helped resolve a real decision.
Admission errors	Unwanted work and missed requests, using reviewed labels.
Follow-up handling	Lost, duplicate, misrouted and out-of-scope requests.
Verified completion	Quality constraint alongside attention and resource measures.

The improvement agent may propose clearer questions, smaller context packs or fewer redundant reports. It cannot suppress required decisions or redefine their outcome labels. Grade admission on labelled source-event cases, separately from repository correctness; observe mode does not establish whether an unexecuted proposal would have succeeded.

39 Intake implementation and acceptance

Keep domain transitions synchronous and testable behind narrow ports. Source adapters translate events; the admission service proposes routing; the existing task service owns task state; native adapters execute; one outbox owns external receipts. This adapts the prototype core and adapter separation [39]. Execution adapters must not write task state directly.

ID	Implementation ticket	Depends on
40	SourceEvent and ConversationSubscription records; web intake and optional GitHub adapter.	01, 14, 20
41	Versioned admission gate, off/observe/auto modes and work mandates.	09, 13, 40
42	Follow-up handler, scoped hand-back and delivery/read/handled receipts.	12, 16, 28, 40, 41
43	PromptBundle composition and default-upgrade comparison.	26, 35, 37
44	Structured human_action cards and decision-first overview.	13, 30, 42
45	Admission dataset and attention metrics in the existing evaluation harness.	31 to 34, 41, 44
46	Source replay, revocation, follow-up and upgrade fault scenarios.	40 to 45

Acceptance scenarios

- Duplicate issue notifications create one admission record; observe mode launches nothing and posts nothing.
- A PR review resumes the correct task; a related thread supplies context without granting scope.
- A crash after delivery but before a read acknowledgement preserves the input and does not silently mark it handled.
- Out-of-scope follow-ups return for admission; a human stop persists through bot feedback and restarts.
- A stale admission result cannot dispatch work after revocation; human follow-ups never bypass global or account limits.
- A changed built-in prompt preserves a custom override and leaves running attempts unchanged; a worse upgrade is rejected by existing evaluations.

Integrate report structure, prompt provenance and attention metrics into the existing milestones. Reserve queue capacity for human follow-ups inside the existing hard limits. Keep the router event-driven and short-lived; technical planning belongs in a separate budgeted architect attempt.

Estimate the optional source-driven intake pilot at 12 to 20 additional engineer-days after the core contracts exist. The existing core estimate remains 100 to 140; including this extension gives 112 to 160. Validate this increment after a source replay and permission spike. No DeepSeek Harness dependency or Feishu connector is required.

40 Platform implementation references

Sources checked on 2 November 2026. Research motivates the design; reported benchmark results are not claims about Mubit repositories. Agent definitions, limits, service targets, promotion rules, staffing and sizing are proposed engineering choices.

[1] Cezar repository

Parallel agent tasks, worktrees, child dispatch and repository license.
https://github.com/open-mercato/cezar

[2] Cezar reference

Workflow gates, fork and join, account profiles and server deployment boundaries.
https://github.com/open-mercato/cezar/blob/main/docs/reference.md

[3] Kandev repository

Multi-agent workbench, remote executors, subtasks and repository license.
https://github.com/kdlbs/kandev

[4] Kandev authentication and team access

Experimental multi-user access and the distinction from filesystem and agent credential isolation.
https://kandev.ai/docs/authentication

[5] Happier repository

Remote agents, native account use, collaboration patterns and repository license.
https://github.com/happier-dev/happier

[6] Happier session sharing

View, edit and manage access; separate delegated approval permission.
https://docs.happier.dev/accounts/session-sharing

[7] Orchestrator repository

Planner and workers, worktrees, CI and review feedback, repository license.
https://github.com/OrchestratorInc/agent-orchestrator

[8] Orchestrator platform support

Desktop and remote access boundaries; shared remote access is not a hosted multi-user service.
https://docs.orchestrator.inc/platforms/

41 Runtime and infrastructure references

[9] Claude Code programmatic operation
Native non-interactive operation, output formats and session behaviour.
https://code.claude.com/docs/en/headless

[10] Claude Code hooks
Lifecycle events, tool decisions, questions and supported permission bridging.
https://code.claude.com/docs/en/hooks

[11] Claude Code authentication and credential use
Distinction between native customer login and authentication in developer products.
https://code.claude.com/docs/en/legal-and-compliance

[12] Codex App Server
Versioned protocol, local stdio transport, generated schemas, streamed events and approvals.
https://learn.chatgpt.com/docs/app-server

[13] OpenCode server
Native local HTTP server, API discovery and event stream.
https://opencode.ai/docs/server/

[14] OpenCode V2 API
Version-specific sessions, permission request/reply routes and event APIs.
https://opencode.ai/v2/docs/api

[15] PostgreSQL explicit locking
Row locking and related database coordination primitives.
https://www.postgresql.org/docs/current/explicit-locking.html

[16] Git worktree reference
Separate working directories within a repository and worktree lifecycle commands.
https://git-scm.com/docs/git-worktree

42 Agent design and evaluation research

[17] Automated Design of Agentic Systems
Meta-agent search over agent designs; research motivation for an explicit design and evaluation loop.
https://arxiv.org/abs/2408.08435

[18] AFlow
Search over agentic workflow structures with execution feedback.
https://arxiv.org/abs/2410.10762

[19] GEPA research paper
Reflective prompt evolution using trajectories and evaluation feedback.
https://proceedings.iclr.cc/paper_files/paper/2026/hash/0e9e708b6f48e14fd0ac29e167413f76-Abstract-Conference.html

[20] GEPA implementation
Local optimisation library with evaluator feedback interfaces; repository lists MIT.
https://github.com/gepa-ai/gepa

[21] Why Do Multi Agent LLM Systems Fail
MAST taxonomy of system design, inter-agent alignment and verification failures.
https://arxiv.org/abs/2503.13657

[22] Anthropic multi agent research system
Dynamic delegation experience and limits of parallelisation for tightly coupled coding tasks.
https://www.anthropic.com/research/multi-agent-research-system

[23] Anthropic agent evaluation engineering
Tasks, trials, outcome and trajectory grading; deterministic, model and human evaluators.
https://www.anthropic.com/research/demystifying-evals-for-ai-agents

[24] Anthropic multi agent coordination research
Coordination and conformity failures motivate measuring the overall system, not just local output.
https://www.anthropic.com/research/multiagent-systems

43 Observability and native agent references

[25] OpenTelemetry agent semantic conventions

Agent, workflow and tool spans. These conventions are in development and should be pinned.
https://github.com/open-telemetry/semantic-conventions-genai/blob/main/docs/gen-ai/gen-ai-agent-spans.md

[26] OpenTelemetry trace concepts

Context, parent-child relationships and span links for asynchronous work.
https://opentelemetry.io/docs/concepts/signals/traces/

[27] MLflow OTLP ingestion

OTLP endpoint with SQL-backed trace storage; supports external instrumentation.
https://mlflow.org/docs/latest/genai/tracing/opentelemetry/ingest/

[28] MLflow self hosting

Self-hosted deployment and storage options. MLflow repository license is Apache-2.0.
https://mlflow.org/docs/latest/self-hosting/

[29] MLflow trace evaluation

Re-score captured execution traces for diagnosis and evaluation.
https://www.mlflow.org/docs/latest/genai/eval-monitor/running-evaluation/traces/

[30] Langfuse agent graph

Graph view inferred from observation nesting and timing; an alternative trace UI.
https://langfuse.com/docs/observability/features/agent-graphs

[31] Langfuse self hosting

Infrastructure requirements include PostgreSQL, ClickHouse, Redis or Valkey and object storage; some add-ons require a license.
https://langfuse.com/self-hosting

[32] Claude Code custom subagents

Native custom instructions, tools, models and child lifecycle. Verify the installed version.
https://code.claude.com/docs/en/sub-agents

[33] Codex subagent configuration

Native roles and subagent settings. Verify supported fields and inherited controls.
https://learn.chatgpt.com/docs/agent-configuration/subagents

[34] OpenCode agent definitions

Version-specific native role configuration and permission behaviour.
https://opencode.ai/v2/docs/agents

44 DeepSeek prototype references

Reviewed at commit 6337fc823ce20251c8de8085948165c81aaa701e on 2 November 2026. Recommendations above are adaptations for our product. No prototype code or runtime dependency has been incorporated.

[35] DeepSeek prototype scope

Research prototype for one repository, packaged for DeepSeek Harness; MIT license.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/README.md

[36] Prototype work authorisation

Admission, mandate scope, observe mode, subscriptions and policy changes.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/docs/002-authorization.md

[37] Prototype execution lifecycle

Follow-ups, background work, reports, worker slots and recovery.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/docs/003-execution.md

[38] Prototype prompt configuration

Layered prompts, default provenance, version conflicts and live reload.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/docs/004-prompts.md

[39] Prototype module boundaries

Domain ports, state ownership, durable effects and documented limitations.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/docs/007-architecture.md

[40] Admission implementation

Intake assessments and stale-policy refusal; separate from output quality grading.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/src/core/evals.ts

[41] Outbound receipt implementation

Atomic receipt creation, deferred delivery and per-conversation ordering.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/src/app/outbox.ts

[42] Executor lifecycle implementation

Worker-slot handling, input-read hooks and assignment recovery.
https://github.com/deepseek-ai/dsh-superagent-prototype/blob/6337fc823ce20251c8de8085948165c81aaa701e/src/executor/dsh.ts